

一、Environment

Window10 / Visual Studio 2022 / glfw 3.3.8

二、Method description

1. OpenGLBufferObject.cpp :

創建、綁定、資料分配、釋放 buffer object，buffer object 是以 id_來創建的，其中資料分配前會先將 VBO 與做綁定，然後再將 data 分配給 buffer object。

A. 將資料分配給綁定的 buffer object。

```
void OpenGLBufferObject::allocateBufferData(const void *data,
                                             GLsizeiptr size) noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glBufferData(type_, size, data, GL_STATIC_DRAW);
}
```

B. 綁定指定 buffer object，同時設置該 buffer 的類型。

```
void OpenGLBufferObject::bind() noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glBindBuffer(type_, id_);
}
```

C. 創建 buffer object

```
void OpenGLBufferObject::create()
{
    PROGRAM_ASSERT(!Detail::isCreated(id_));

    // Fill in the Blank
    glGenBuffers(1, &id_);

    if (!Detail::isCreated(id_))
    {
        throw OpenGLException("OpenGLBufferObject failed to instantiate.");
    }
}
```

D. 取消綁定當前 bind 的 VBO。

```
void OpenGLBufferObject::release() noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glBindBuffer(type_, 0);
}
```

E. 釋放指定的 buffer object。

```
void OpenGLBufferObject::tidy() noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glDeleteBuffers(1, &id_);

    id_ = Detail::noId;
}
```

2. OpenGLShader.cpp

創建、編譯、釋放 shader 及將 shader source 綁定在 shader 上對模型上色。

A. 檢查 shader 是否成功編譯。

```

inline bool compileStatus(GLuint id) noexcept
{
    GLint status;
    // Fill in the Blank
    glGetShaderiv(id, GL_COMPILE_STATUS, &status);

    return (status == GL_TRUE);
}

```

B. 編譯給予的 shader source。

```

bool OpenGLShader::compileFromSource(const char *source) noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glShaderSource(id_, 1, &source, NULL);
    // Fill in the Blank
    glCompileShader(id_);

    return Detail::compileStatus(id_);
}

```

C. 創建 shader。

```

void OpenGLShader::create()
{
    PROGRAM_ASSERT(!Detail::isCreated(id_));

    // Fill in the Blank
    id_ = glCreateShader(type_);

    if (!Detail::isCreated(id_))
    {
        throw OpenGLException("OpenGLShader failed to instantiate.");
    }
}

```

D. 釋放 shader。

```

void OpenGLShader::tidy() noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glDeleteShader(id_);

    id_ = Detail::noId;
}

```

3. OpenGLShaderProgram.cpp

創建 shader program 將 shader 連結起來。

A. 將一個 shader attach 到 shader program 中

```

void OpenGLShaderProgram::attachShader(
    std::unique_ptr<OpenGLShader> &&shader) noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glAttachShader(id_, shader->id());

    shaders_.push_back(std::move(shader));
}

```

B. 創建 shader program

```

void OpenGLShaderProgram::create()
{
    PROGRAM_ASSERT(!Detail::isCreated(id_));

    // Fill in the Blank
    id_ = glCreateProgram();

    if (!Detail::isCreated(id_))
    {
        throw OpenGLException("OpenGLShaderProgram failed to instantiate.");
    }
}

```

C. 釋放 shader program

```
void OpenGLShaderProgram::destroyProgram() noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glDeleteShader(id_);

    id_ = Detail::noId;
}
```

D. ——釋放 shader program 中的 shaders

```
void OpenGLShaderProgram::destroyShaders() noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    for (auto &shader : shaders_)
    {
        // Fill in the Blank
        glDeleteShader(shader->id());

        shader.reset(nullptr);
    }

    shaders_.clear();
}
```

E. 禁用、啟用 vertex attribute array。

```
void OpenGLShaderProgram::disableVertexAttribArray(GLuint index) noexcept
{
    // Fill in the Blank
    glDisableVertexAttribArray(index);
}

void OpenGLShaderProgram::enableVertexAttribArray(GLuint index) noexcept
{
    // Fill in the Blank
    glEnableVertexAttribArray(index);
}
```

F. 連接 shader program 中的 shader，及檢查連接狀態。

```
void OpenGLShaderProgram::link() noexcept
{
    // Fill in the Blank
    glLinkProgram(id_);
}

bool OpenGLShaderProgram::linkStatus() const noexcept
{
    GLint status;
    // Fill in the Blank
    glGetProgramiv(id_, GL_LINK_STATUS, &status);

    return (status == GL_TRUE);
}
```

G. 將 vertex attribute 映射到 shader program。

```
void OpenGLShaderProgram::mapAttributePointer(GLuint index, GLint size,
                                              GLenum type, GLboolean normalized,
                                              GLsizei stride,
                                              GLint offset) noexcept
{
    // Fill in the Blank
    glVertexAttribPointer(index, size, type, normalized, stride,
                          (void *) (offset * sizeof(float)));
}
```

H. 使用該 shader program

```
void OpenGLShaderProgram::use() noexcept
{
    // Fill in the Blank
    glUseProgram(id_);
}
```

4. OpenGLTexture.cpp :

將圖片貼在 3D 的物體(茶壺)上。

A. 綁定 texture object。

```
void OpenGLTexture::bind()
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glBindTexture(GL_TEXTURE_2D, id_);
}
```

B. 將 texture 數據從 buffer 複製到要 texture 的 object。

```
void OpenGLTexture::bindBuffer(const std::vector<unsigned char> &buffer)
{
    // Fill in the Blank
    // (bind)
    glBindBuffer(GL_ARRAY_BUFFER, id_);

    // (parameter setup: filter and wrapping method)
    setMinificationFilter(minificationFilter_);
    setMagnificationFilter(magnificationFilter_);
    setWrapOption(wrapOption_);

    // (data specify)
    glTexImage2D(GL_TEXTURE_2D, 0, format_, width_, height_, 0, GL_RGB,
                GL_UNSIGNED_BYTE, &buffer[0]);
    // (generate mipmap)
    glGenerateMipmap(GL_TEXTURE_2D);
}
```

C. 創建 texture object。

```
void OpenGLTexture::create()
{
    PROGRAM_ASSERT(!Detail::isCreated(id_));

    // Fill in the Blank
    glGenTextures(1, &id_);

    if (!Detail::isCreated(id_))
    {
        throw OpenGLException(
            "OpenGLTexture instantiate failed at 'glGenTextures'.");
    }
}
```

D. 釋放 texture object。

```
void OpenGLTexture::release()
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glBindTexture(GL_TEXTURE_2D, 0);
}
```

E. 分別是設置 texture 的放大、縮小及包覆方式。

```
void OpenGLTexture::setMagnificationFilter(Filter filter)
{
    PROGRAM_ASSERT(Detail::isCreated(id_));
    magnificationFilter_ = filter;

    bind();
    // Fill in the Blank
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, magnificationFilter_);
    release();
}

void OpenGLTexture::setMinificationFilter(Filter filter)
{
    PROGRAM_ASSERT(Detail::isCreated(id_));
    minificationFilter_ = filter;

    bind();
    // Fill in the Blank
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, minificationFilter_);
    release();
}

void OpenGLTexture::setWrapOption(WrapOption option)
{
    PROGRAM_ASSERT(Detail::isCreated(id_));
    wrapOption_ = option;

    bind();
    // Fill in the Blank
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, wrapOption_);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, wrapOption_);
    release();
}
```

F. 刪除 texture object。

```
void OpenGLTexture::tidy()
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glDeleteTextures(1, &id_);

    id_ = 0;
}
```

5. OpenGLVertexArrayObject.cpp :

創建、綁定、釋放 vertex array object，將 id_ 設置成該 VAO 的唯一編號。

A. 使用來綁定特定的 VAO，id_ 是該 VAO 的編號。

```
void OpenGLVertexArrayObject::bind() noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glBindVertexArray(id_);
}
```

B. 取消綁定當前 bind 的 VAO。

```
void OpenGLVertexArrayObject::release() noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glBindVertexArray(0);
}
```

C. 生成 VAO，將其編號存至 id_ 變數中。

```
void OpenGLVertexArrayObject::create()
{
    PROGRAM_ASSERT(!Detail::isCreated(id_));

    // Fill in the Blank
    glGenVertexArrays(1, &id_);

    if (!Detail::isCreated(id_))
    {
        throw OpenGLException("OpenGLVertexArrayObject failed to instantiate.");
    }
}
```

D. 刪除指定的 VAO

```
void OpenGLVertexArrayObject::tidy() noexcept
{
    PROGRAM_ASSERT(Detail::isCreated(id_));

    // Fill in the Blank
    glDeleteVertexArrays(1, &id_);

    id_ = Detail::noId;
}
```

三、How to run the program

1. 在終端機輸入：

```
PS D:\課程\電腦繪圖\HW1\76121314_夢煜璽\sample_code\build\bin\Release> .\Homework01.exe "resources/model/Utah_teapot_(solid)_texture.obj" "resources/texture/uv.png" "Shader/BasicVertexShader.vs.glsl" "Shader/BasicFragmentShader.fs.glsl"
```

一開始會先建立 shader program 與 shader，每次建立完 shader 會將其 attach 到 shader program 中，然後將裡面的 shader 做連結。之後會創建 texture，然後通過 bind 和 release，設置 Minification Filter、Magnification Filter 和 WrapOption，之後

創建 VAO、VBO 並將它們設置好後，通過創建 mesh 來繪圖，最後再將使用完畢的 mesh、texture 和 shader 做 tidy 刪除。